In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Step 2: Load the dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passe
df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")

print("First five rows:")
print(df.head())

print("\nDataset information:")
print(df.info())

# Step 3: Explore the dataset
print("\nShape:", df.shape)
print("\nStatistical summary:")
print(df.describe())

print("\nMissing values:")
print(df.isnull().sum())

# Step 4: Plot the original time series
plt.figure(figsize=(12, 5))
plt.plot(df["Passengers"], label="Monthly Passengers")
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.show()


# Step 5: Augmented Dickey-Fuller (ADF) stationarity test
def adf_test(series, title):
    result = adfuller(series.dropna())

    print(f"\n{title}")
    print("ADF Statistic:", result[0])
    print("p-value:", result[1])

    if result[1] <= 0.05:
        print("Result: Stationary")
    else:
        print("Result: Non-stationary")


adf_test(df["Passengers"], "ADF Test on Original Series")


# Step 6: Apply log transformation and differencing
# Log transformation helps stabilize increasing variance.
# First differencing removes much of the trend.
```

```
df["Log_Passengers"] = np.log(df["Passengers"])
df["Log_Diff"] = df["Log_Passengers"].diff()

print("\nTransformed DataFrame:")
print(df.head(10))

# Step 7: Re-run ADF test on differenced series
adf_test(df["Log_Diff"], "ADF Test on Log-Differenced Series")

# Visualize the transformed series
plt.figure(figsize=(12, 5))
plt.plot(df["Log_Diff"], label="Log-Differenced Passengers")
plt.title("Log-Differenced Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Change in Log Passengers")
plt.legend()
plt.grid(True)
plt.show()


# Step 8: Select the differencing order
# Use d = 1 for ARIMA because the model uses
# the log-transformed series with first differencing.
d = 1
print("\nSelected differencing order d =", d)


# Step 9: Split the original series into training and testing sets
series = df["Passengers"]

split = int(len(series) * 0.80)

train = series.iloc[:split]
test = series.iloc[split:]

print("\nTraining size:", len(train))
print("Testing size:", len(test))
print("Training period:", train.index.min(), "to", train.index.max())
print("Testing period:", test.index.min(), "to", test.index.max())


# Step 10: Fit ARIMA model on training data
# Fit on log-transformed passenger numbers.
# ARIMA(2, 1, 2): p=2, d=1, q=2

train_log = np.log(train)

model = ARIMA(train_log, order=(2, d, 2))
model_fit = model.fit()

print("\nARIMA Model Summary:")
print(model_fit.summary())

# Step 11: Forecast passenger numbers for the test period
forecast_log = model_fit.forecast(steps=len(test))

# Convert forecasts back to passenger counts
predictions = np.exp(forecast_log)

# Keep predictions aligned with test dates
```

```
predictions = pd.Series(
    predictions.to_numpy(),
    index=test.index,
    name="Predicted Passengers"
)

print("\nForecasted Passenger Numbers:")
print(predictions)


# Step 12: Evaluate forecast accuracy
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)

print("\nForecast Evaluation Metrics:")
print("MSE :", round(mse, 2))
print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))


# Step 13: Plot training, testing, and predictions
plt.figure(figsize=(14, 6))

plt.plot(train, label="Training Data")
plt.plot(test, label="Actual Test Data")
plt.plot(predictions, label="ARIMA Forecast", linestyle="--")

plt.title("Airline Passenger Forecast using ARIMA")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.show()
```

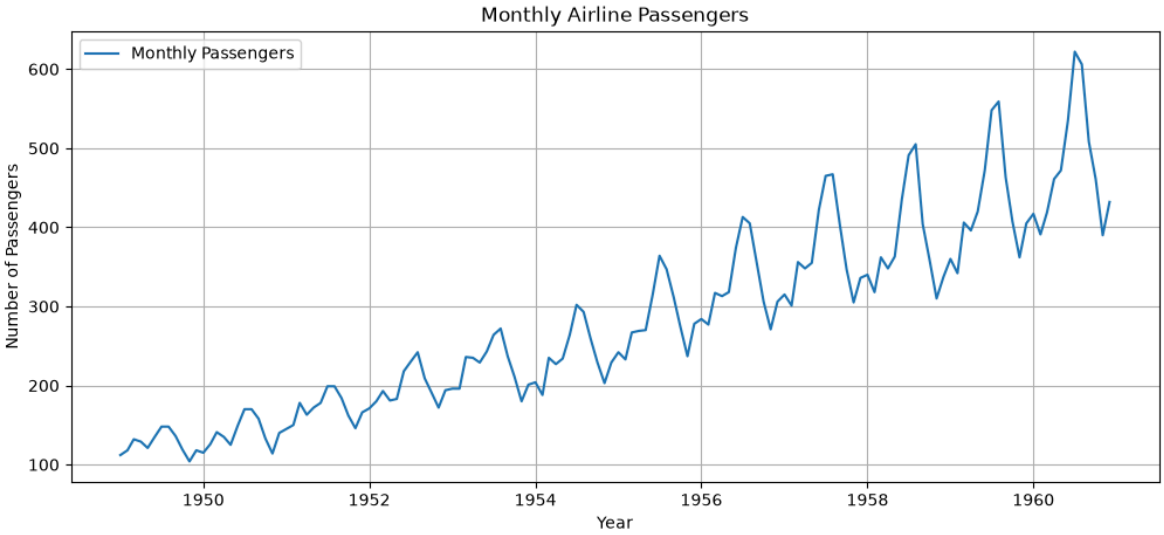
```
First five rows:
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121

Dataset information:
<class 'pandas.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None

Shape: (144, 1)

Statistical summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000

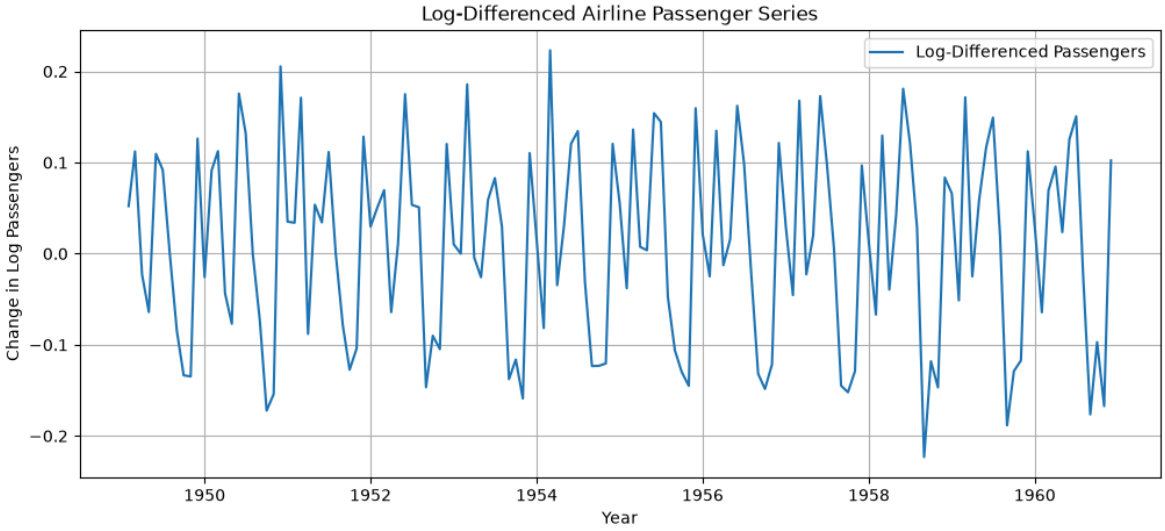
Missing values:
Passengers    0
dtype: int64
```



```
ADF Test on Original Series
ADF Statistic: 0.8153688792060472
p-value: 0.991880243437641
Result: Non-stationary

Transformed DataFrame:
            Passengers  Log_Passengers  Log_Diff
Month
1949-01-01         112        4.718499       NaN
1949-02-01         118        4.770685  0.052186
1949-03-01         132        4.882802  0.112117
1949-04-01         129        4.859812 -0.022990
1949-05-01         121        4.795791 -0.064022
1949-06-01         135        4.905275  0.109484
1949-07-01         148        4.997212  0.091937
1949-08-01         148        4.997212  0.000000
1949-09-01         136        4.912655 -0.084557
1949-10-01         119        4.779123 -0.133531

ADF Test on Log-Differenced Series
ADF Statistic: -2.717130598388141
p-value: 0.07112054815085739
Result: Non-stationary
```




```
Selected differencing order d = 1

Training size: 115
Testing size: 29
Training period: 1949-01-01 00:00:00 to 1958-07-01 00:00:00
Testing period: 1958-08-01 00:00:00 to 1960-12-01 00:00:00
C:\Users\rena\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
C:\Users\rena\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
C:\Users\rena\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473:
ValueWarning: No frequency information was provided, so inferred frequency MS wil
l be used.
  self._init_dates(dates, freq)
```

ARIMA Model Summary:

SARIMAX Results

Dep. Variable:	Passengers	No. Observations:	115
Model:	ARIMA(2, 1, 2)	Log Likelihood	104.653
Date:	Fri, 09 Oct 2026	AIC	-199.305
Time:	11:38:37	BIC	-185.624
Sample:	01-01-1949 - 07-01-1958	HQIC	-193.753
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.2477	0.188	1.321	0.186	-0.120	0.615
ar.L2	0.2583	0.241	1.069	0.285	-0.215	0.732
ma.L1	-0.0887	0.141	-0.627	0.530	-0.366	0.188
ma.L2	-0.6558	0.175	-3.738	0.000	-1.000	-0.312
sigma2	0.0093	0.002	4.915	0.000	0.006	0.013

Ljung-Box (L1) (Q):	0.03	Jarque-Bera (JB):	6.52
Prob(Q):	0.87	Prob(JB):	0.04
Heteroskedasticity (H):	1.06	Skew:	-0.00
Prob(H) (two-sided):	0.85	Kurtosis:	1.83

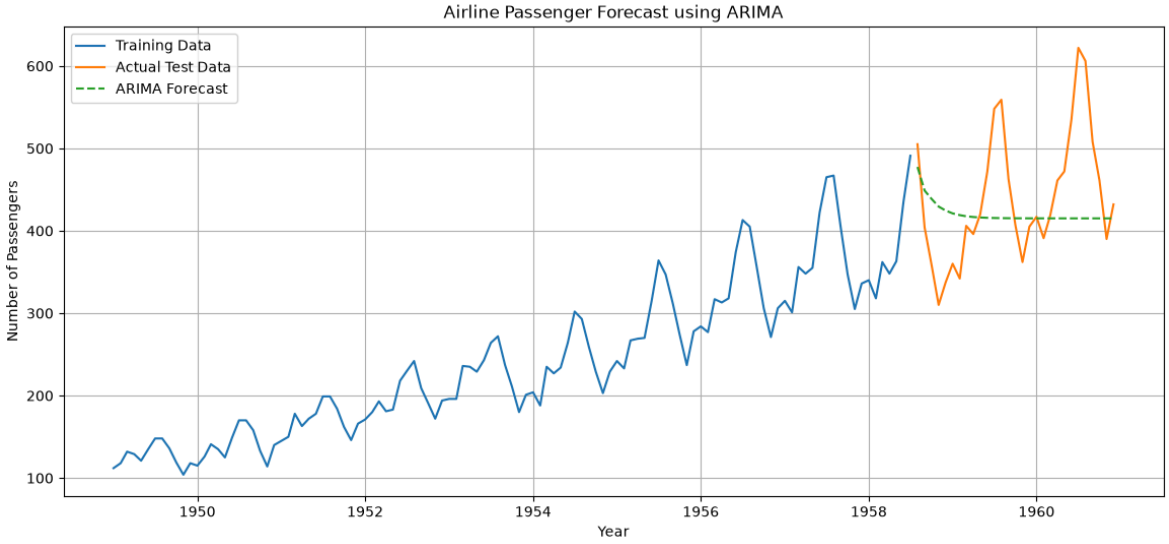
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Forecasted Passenger Numbers:

Month	
1958-08-01	477.409529
1958-09-01	448.755904
1958-10-01	438.734989
1958-11-01	429.368664
1958-12-01	424.595667
1959-01-01	421.068233
1959-02-01	418.987541
1959-03-01	417.573034
1959-04-01	416.689916
1959-05-01	416.107854
1959-06-01	415.736409
1959-07-01	415.494416
1959-08-01	415.338681
1959-09-01	415.237664
1959-10-01	415.172444
1959-11-01	415.130209
1959-12-01	415.102907
1960-01-01	415.085237
1960-02-01	415.073810
1960-03-01	415.066416
1960-04-01	415.061633
1960-05-01	415.058538
1960-06-01	415.056536

```
1960-07-01    415.055241
1960-08-01    415.054403
1960-09-01    415.053861
1960-10-01    415.053511
1960-11-01    415.053284
1960-12-01    415.053137
Name: Predicted Passengers, dtype: float64

Forecast Evaluation Metrics:
MSE : 6869.01
MAE : 62.63
RMSE: 82.88
```



In [ ]: